University of Waterloo
Faculty of Engineering
Department of Electrical and Computer Engineering

ECE 206
Laboratory 1

Prepared by
Lo, Ethan
UW Student ID Number: 21124050
UW User ID: e33lo @uwaterloo.ca

Prepared by
Lepage, Yohann
UW Student ID Number: 21124488
UW User ID: ylepage@uwaterloo.ca

Prepared by
Teeter, Jason
UW Student ID Number: 21157226
UW User ID: jteeter@uwaterloo.ca

Prepared by
Du, Nicholas
UW Student ID Number: 21031714
UW User ID: n4du@uwaterloo.ca

2B Nanotechnolgy Engineering
20 May 2026

Having watched the presentation, we will go through the following exercise:

1.1 Stating the Problem

Summarize in a sentence or two what you are trying to approximate in a boundary-value problem (BVP) with a linear ordinary differential equation (LODE) and a forcing function $g(x)$.

You are trying to approximate a function y(x) that satisfies the differential equation and takes the required values at the boundaries.

1.2 Determining the Formal Parameters

Suppose we want to solve a boundary value problem where the ordinary differential equation is known to be linear ODE with constant coefficients together with a forcing function $g(x)$; that is, we wish to approximate $u(x)$ when

$$c_1 u^{(2)}(x) + c_2 u^{(1)}(x) + c_3 u(x) = g(x)$$
$$u(a) = u_a$$
$$u(b) = u_b$$

What are the values that would have to be passed by the user to a function? Answer this question by filling in Tables 1.1b.i, 1.1b.ii, and 1.1b.iii. Note the difference between a parameter that is used to describe the BVP (Tables 1.1b.i and 1.1b.ii) and a parameter that is used to control the numerical solver (Table1.1b.iii). The parameter(s) describing the problem should come before the parameter(s) used in the numerical solution when calling the function.

Table 1.1b.i Parameters describing the problem.

Given Values	Description
$[c_1, c_2, c_3]$	The coefficients of the ODE, c1 being the one on u''(x)
$[a, b]$	The interval over which the solution is defined
$[u_a, u_b]$	Boundary conditions at x=a and x=b

Table 1.1b.ii Functions describing the problem.

Given Functions	Arguments	Description
g	x	The forcing function of the ODE

Table 1.1b.iii Parameters that control the numerical solver.

Method Parameters	Description
n	Number of points on the interval

Comments:

1. You can add more rows to the tables.
2. The descriptions should be independent of any programming language.
3. Recall that in Matlab, when a function is passed as an argument, you must pass it with `@f` which creates a *function handle*.

1.3 Determining the Return Values

The return values should included two vectors:

1. A vector of x values, and
2. A vector of u values.

The order of the output should be the same as the output for the Matlab function `ode45` (see `help ode45`). Give a brief description of these n-dimensional vectors.

Symbol	Description
$\mathbf{x}_{out}$	The vector $\mathbf{x}_{out}$ is an n-dimensional vector of x points at which the solution was approximated
$\mathbf{u}_{out}$	The vector $\mathbf{u}_{out}$ is an n-dimensional vector containing the approximate value of u(x) at the corresponding point in xout

1.4 The Signature and Description of the Matlab Function

Given the responses in 1.1, 1.2 and 1.3, fill in the description of the function here.

```
% bvp
% approximate a function y(x) that satisfies the differential equation and
% takes the required values at the boundaries.
%
% Parameters
% ==========
%    c:       The coefficients of the ODE, c1 being the one on u''(x)
%    x_int:   The interval over which the solution is defined
%    u_int:   Boundary conditions at x=a and x=b
%
%    g:       The forcing function for the ODE
%
%    n:       Number of points on the interval
%
% Return Values
% =============
%    x:       vector of x points at which the solution was approximated
%    u:       vector of approximate values of u(x) at the point in x

function [x, u] = bvp( c, x_int, u_int, g, n )
```

Save this function in Matlab and remember that it must be called `bvp.m`. To create a new function in Matlab, use *File→New→Function*. Make sure that the directory that you are saving the function in is the *Current Folder*.

1.5 Argument Checking

The next step is to check the dimensions of the arguments. The function `size( w )` returns a vector containing the row and column dimensions. This is then checked against a vector of the expected dimension in order to ensure that `w` is a 3D column vector (a 3 × 1 matrix). The function `isscalar( x )` returns true if `x` is a 1 × 1 matrix (Matlab's version of a scalar value). A scalar is an integer if it equals itself when rounded. Finally, the `isa` command can be used to determine other properties. Replace the code below with your argument-checking conditional statements. Some examples of argument checking are provided.

```
    if ~all( size( w ) == [3, 1] )
        throw( MException( 'MATLAB:invalid_argument', ...
        'the argument w is not a 3-dimensional column vector' ) );
    end

    if ~isscalar( x )
        throw( MException( 'MATLAB:invalid_argument', ...
        'the argument x is not a scalar' ) );
    end

    if ~isscalar( y ) || ( y ~= round( y ) )
        throw( MException( 'MATLAB:invalid_argument', ...
        'the argument y is not an integer' ) );
    end

    if ~isa( z, 'function_handle' )
        throw( MException( 'MATLAB:invalid_argument', ...
        'the argument z is not a function handle' ) );
    end
```

```
function [x, u] = bvp( c, x_int, u_int, g, n )

    % Argument Checking

    % Check c is a 3x1 column vector
    if ~all(size(c) == [3, 1])
        throw(MException('MATLAB:invalid_argument', ...
            'the argument c must be a 3x1 column vector'));
    end

    % Check x_int is a 2-element vector
    if ~isvector(x_int) || length(x_int) ~= 2
        throw(MException('MATLAB:invalid_argument', ...
            'the argument x_int must be a 2-element vector [a, b]'));
    end

    % Check u_int is a 2-element vector
    if ~isvector(u_int) || length(u_int) ~= 2
        throw(MException('MATLAB:invalid_argument', ...
```

```
        'the argument u_int must be a 2-element vector [ua, ub]'));
end

% Check g is a function handle
if ~isa(g, 'function_handle')
    throw(MException('MATLAB:invalid_argument', ...
        'the argument g must be a function handle'));
end

% Check n is a positive integer scalar
if ~isscalar(n) || n ~= round(n) || n <= 0
    throw(MException('MATLAB:invalid_argument', ...
        'the argument n must be a positive integer scalar'));
end
```

1.6 Describing the Solution

We will now continue with the development of this function that you have defined. Write down, in English, the steps that you would perform in order to solve a boundary-value problem. Points to remember:

1. The output vector of x values will require n entries; however, the system of equations we are solving will be $(n-2) \times (n-2)$. Which $n-2$ variables will be found by solving this system of equation?
2. What additional values are required in creating the matrix and the right-hand vector?
3. Once you have solved for the n-2 unknown u values, can we simply pass these values back to the user?

1. First step: divide the interval [a,b] into n points, find step size h = (b-a)/(n-1)
2. Second step: use finite differences to approximate derivatives in the ODE
3. Make a matrix for the unknowns and RHS vector with g(x), include BCs in RHS
4. Solve the equation to find the values of u
5. Add BCs to the values
6. Return values

Remember to describe your steps in English. It would make sense to step through the slides to determine the steps. Do not combine too many operations into a single step.

1.7*a* Implementing the Solution

The next step is to convert your code to Matlab code. Each of the steps you indicated in Question 1.2*a* will be translated into one or more Matlab commands. For each step, indicate the appropriate Matlab commands. If you define a local variable in any step, document the purpose of the variable.

```
% Step 1:  give a brief description
a = ...;    % purpose
Your commands here
% Step 2:  give a brief description
b = ...;    % purpose
Your commands here, ...
```

Copy and paste your entire Matlab function with comments here:

```
% Step 1: Grid setup
a = x_int(1);
b = x_int(2);
ua = u_int(1);
ub = u_int(2);

x = linspace(a, b, n)'; % column vector of grid points
h = (b - a) / (n - 1); % step size

% Step 2: Build system of equations for interior points
% Number of unknowns
m = n - 2;

% Coefficients for finite difference
c1 = c(1); % coefficient of u''
c2 = c(2); % coefficient of u'
c3 = c(3); % coefficient of u

% Initialize matrix A and right-hand side vector F
A = zeros(m, m);
F = zeros(m, 1);

for i = 1:m
xi = x(i+1); % interior point

% Finite difference approximations
% u'' ~ (u_{i-1} - 2*u_i + u_{i+1}) / h^2
% u' ~ (u_{i+1} - u_{i-1}) / (2*h)

% Diagonal entries
if i > 1
A(i, i-1) = c1 / h^2 - c2 / (2*h); % left neighbor
end
A(i, i) = -2 * c1 / h^2 + c3; % center
if i < m
```

```
A(i, i+1) = c1 / h^2 + c2 / (2*h); % right neighbor
end

% Right-hand side
F(i) = g(xi);

% Adjust for known boundary values
if i == 1
F(i) = F(i) - (c1 / h^2 - c2 / (2*h)) * ua;
end
if i == m
F(i) = F(i) - (c1 / h^2 + c2 / (2*h)) * ub;
end
end

% Step 3: Solve the linear system
u_interior = A \ F; % solution at interior points

% Step 4: Assemble full solution
u = [ua; u_interior; ub];

end
```

1.8 Testing your Implementation

We are now ready to test your code.

1.8*a* In the Laboratory slides, two examples are provided. The first example is homogeneous: that is, the forcing function is zero. You will therefore have to write a forcing function that takes a vector `x` as an argument and return a vector of zeros of the same dimension as `x` (try `y = 0*x`). The second has a non-zero forcing function that you will also have to implement. Denote these two forcing functions by `g1` and `g2` respectively. With these functions, you will determine whether or not your code is working. Plot the points. Your plots do not have to include the actual solutions (the functions indicated in the slides in red). Instead, you will only have to plot the points as circles. For each of the outputs, include the title with your UW User ID(s).

```
title( 'uwuserid' );
title( 'uwuserid and uwuserid' );
```

Save these two images and replace them in Figures 1 and 2. List the commands you used to:

1. Call your function and approximate the solution, and
2. Plot the two approximations.

```
g1 = @(x) 0 * x;

g2 = @(x) -4 * sin(2 * pi * x);

%bvp1

[x1, u1] = bvp([1; 3; 2], [0, 1], [4, 5], g1, 9);


figure(1)

plot(x1, u1, 'o')

xlabel('x')

ylabel('u')

title('n4du, e33lo, ylepage, jteeter')


%bvp2
```

```
[x2, u2] = bvp([1; 3; 2], [0, 1], [4, 5], g2, 9);


figure(2)

plot(x2, u2, 'o')

xlabel('x')

ylabel('u')

title('n4du, e33lo, ylepage, jteeter')
```

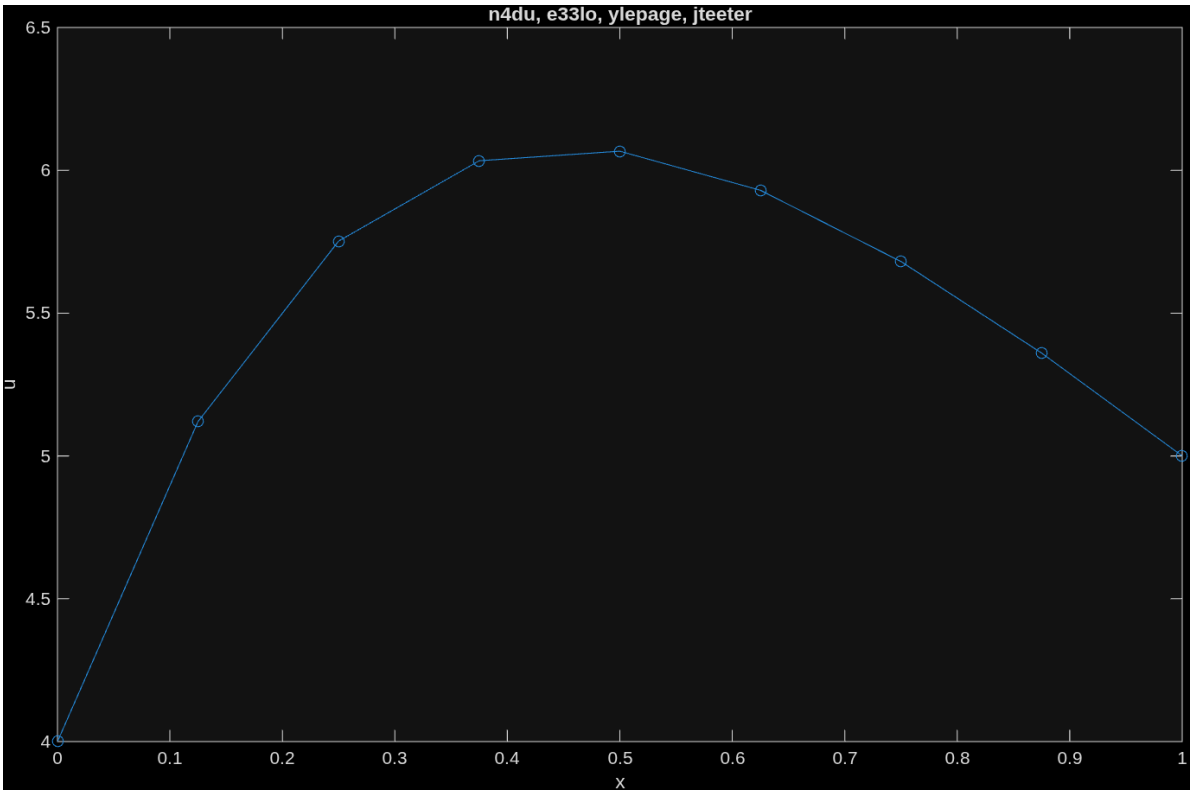

Figure 1. The first solution.

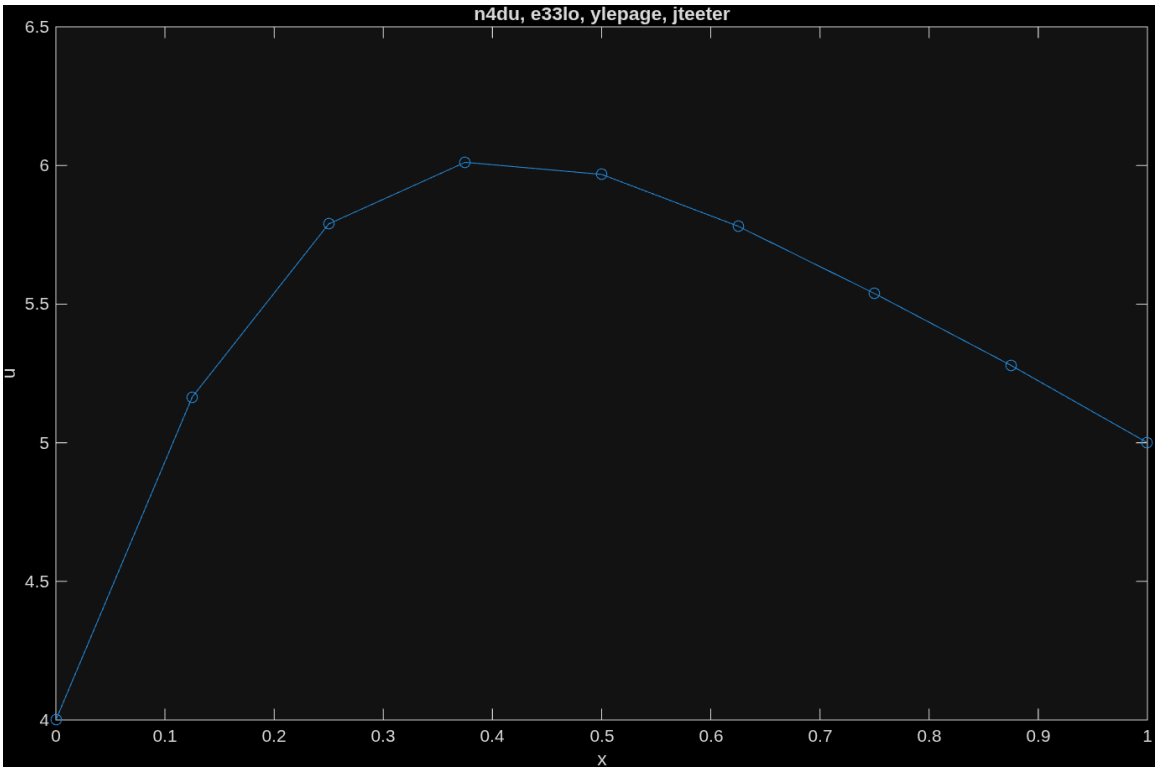

Figure 2. The second solution.

1.8*b* Repeat Problem 1.8*a*, but use a larger number of points. Use a value of n equal to 50 plus the last two digits of one of your UW Student ID numbers. Do not use circles, as you did in the previous question; instead, use red points in your plot. Use `help plot` to see how to plot points instead of circles. Save these two images and replace them in Figures 3 and 4. Be sure to include your UW User ID(s) in the title.

Save these two images and replace them in Figures 1 and 2. List the commands you used to solve the problem and plot the two solutions.

```
I = 17;

n = 50 + I;


g1 = @(x) 0 * x;

g2 = @(x) -4 * sin(2 * pi * x);

%bvp1

[x1, u1] = bvp([1; 3; 2], [0, 1], [4, 5], g1, n);


figure(1)

plot(x1, u1, 'r.', 'MarkerSize', 12)

xlabel('x')

ylabel('u')

title('n4du, e33lo, ylepage, jteeter')

%bvp2

[x2, u2] = bvp([1; 3; 2], [0, 1], [4, 5], g2, n);


figure(2)

plot(x2, u2, 'r.', 'MarkerSize', 12)

xlabel('x')

ylabel('u')

title('n4du, e33lo, ylepage, jteeter')
```

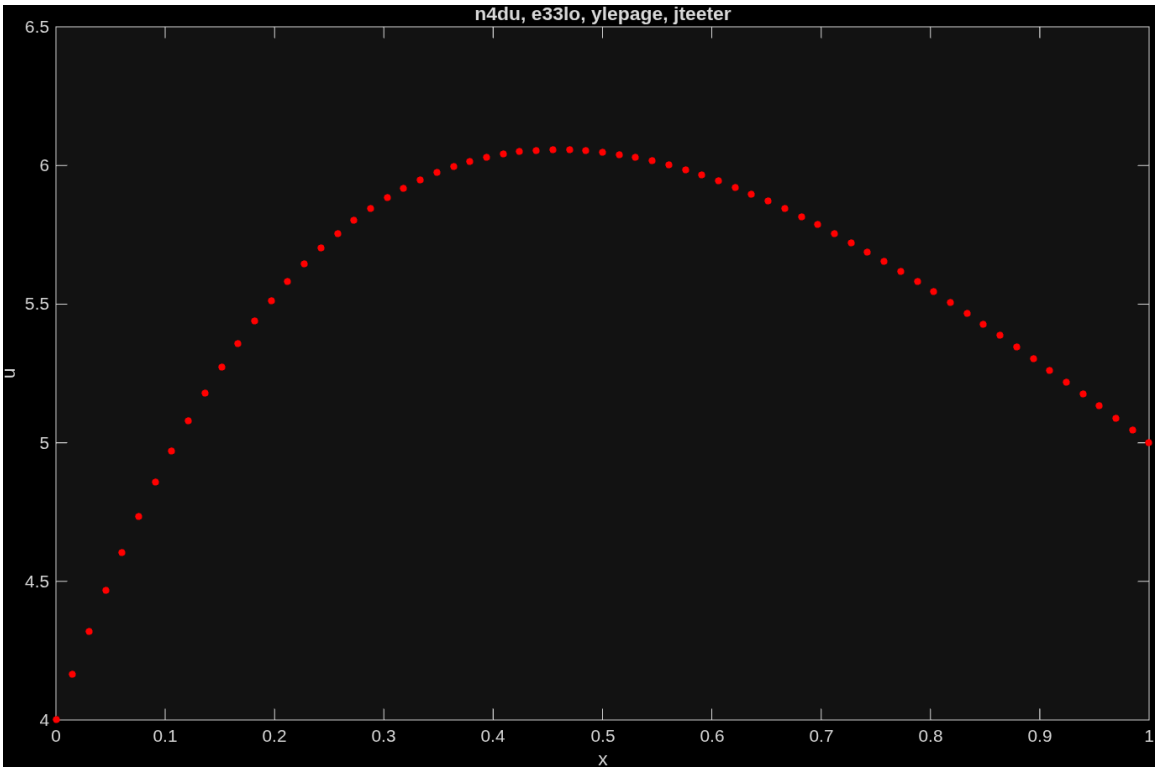

Figure 3. The first solution.

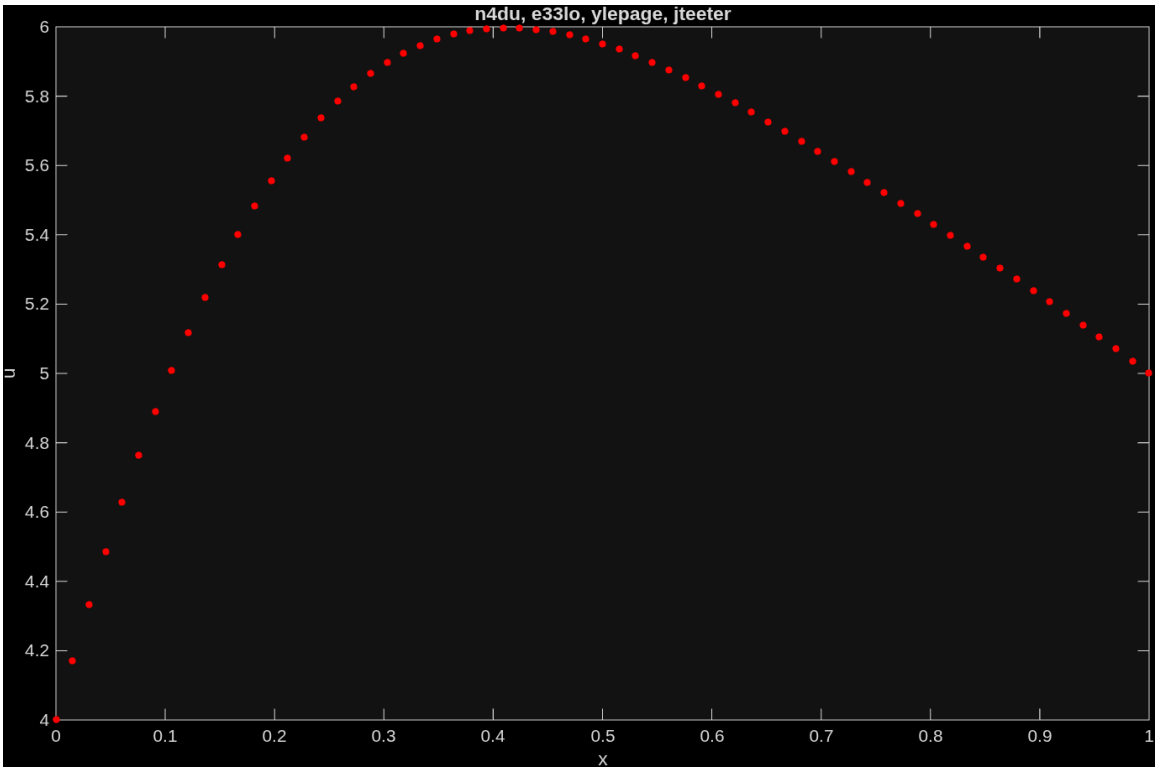

Figure 4. The second solution.

1.8*c* Now is your opportunity to create and solve your own BVP. Select values where the coefficients of the ODE are non-zero, pick two boundary points a and b where $a < b$ and pick two boundary values. Use Maple to generate the solution by using the two commands:

```
dsolve( {c1*(D@@2)(u)(x) + c2*D(u)(x) + c3*u(x) = 0, u(a) = u_a, u(b) = u_b}, u(x) );
plot( rhs( % ), x = a..b );   # use the same a and b...
```

Launch Maple and select *File→New→Worksheet Mode* and cut-and-paste the above Maple command.

Make sure you replace anything in red in the Maple code with your numbers!

Indicate the variables you have chosen here:

$c_1 = 3$
$c_2 = 1$
$c_3 = 4$
$a = 1$
$b = 5$
$u_a = 9$
$u_b = 2$

Copy the plot from Maple into Figure 5.

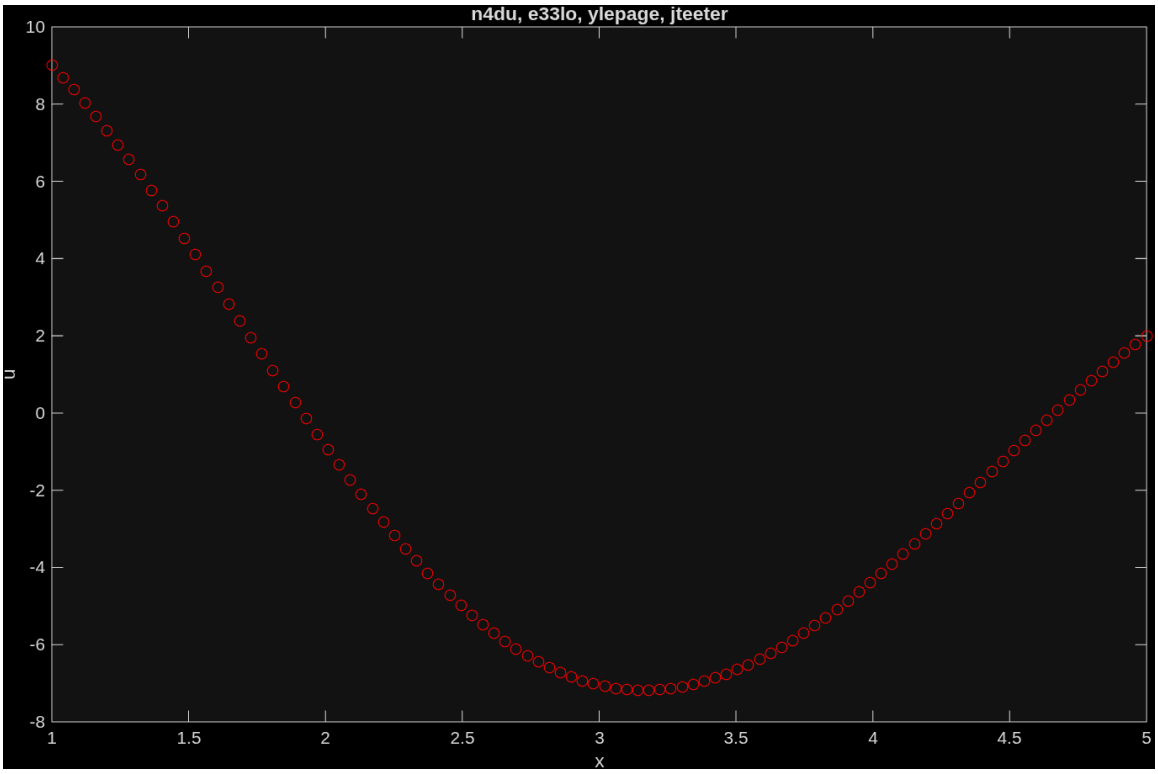

Figure 5. The Maple solution.

Finally, use your function with $n = 10$ points (use red circles) and $n = 100$ points (use red points) and copy the Matlab plots into Figure 6 and 7. Be sure to include your UW User ID(s) in the title.

List the commands you used to solve the problem and plot the two solutions.

```
% ODE Coefficients
c1 = 3;
c2 = 1;
c3 = 4;
a = 1;
b = 5;
u_g = 9;
u_b = 2;

f = @(x) exp(sin(x));

n = 100;

% Matlab built in solution (bvp5c):
sol = bvp4c(@(x, y) [y(2); (-c2 * y(2) - c3 * y(1)) / c1], @(ya, yb) [ya(1) - u_g; yb(1) - u_b],
bvpinit(linspace(a, b, 25), [u_g; 0]));

x = linspace(a, b, n);
u = deval(sol, x);
u = u(1, :)';

% [x, u] = bvp([c1; c2; c3], [a, b], [u_g, u_b], f, 10);

figure(1)

plot(x, u, 'ro');
xlabel('x');
ylabel('u');
title('n4du, e33lo, ylepage, jteeter')

% Assignment solution:
[x, u] = bvp([c1; c2; c3], [a, b], [u_g, u_b], f, 100);

figure(2)

plot(x, u, 'r.', 'MarkerSize', 8);
```

```
xlabel('x');
ylabel('u');
title('n4du, e33lo, ylepage, jteeter')
```

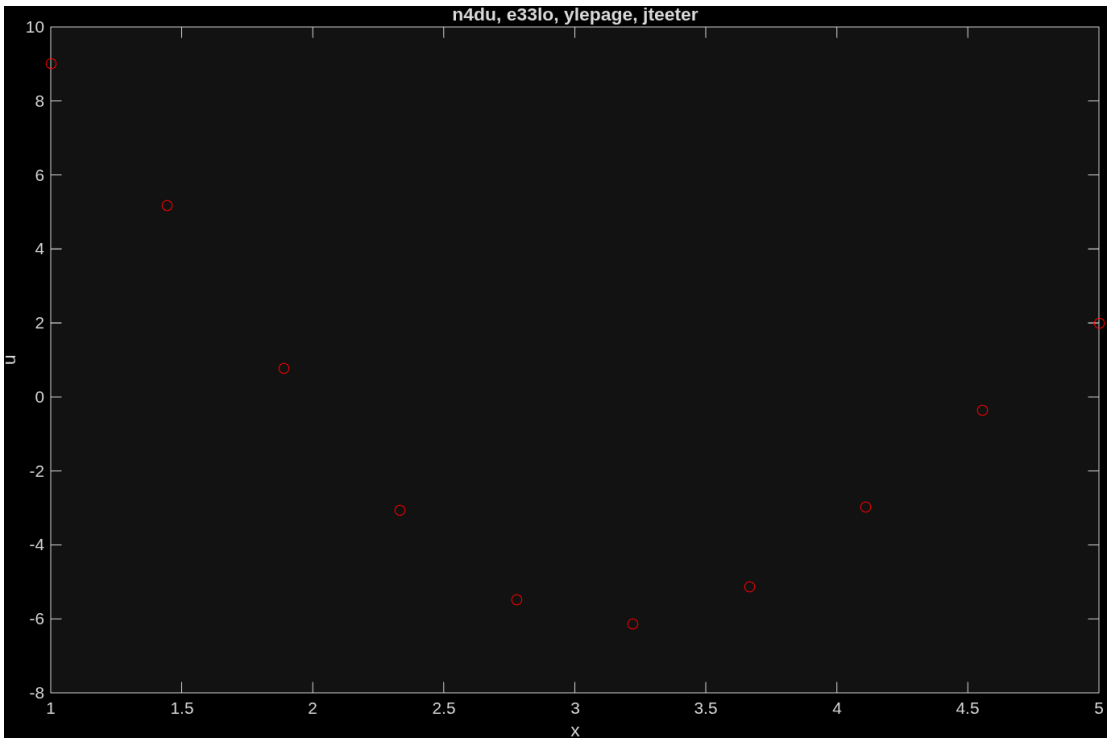

Figure 6. The approximation with 10 points.

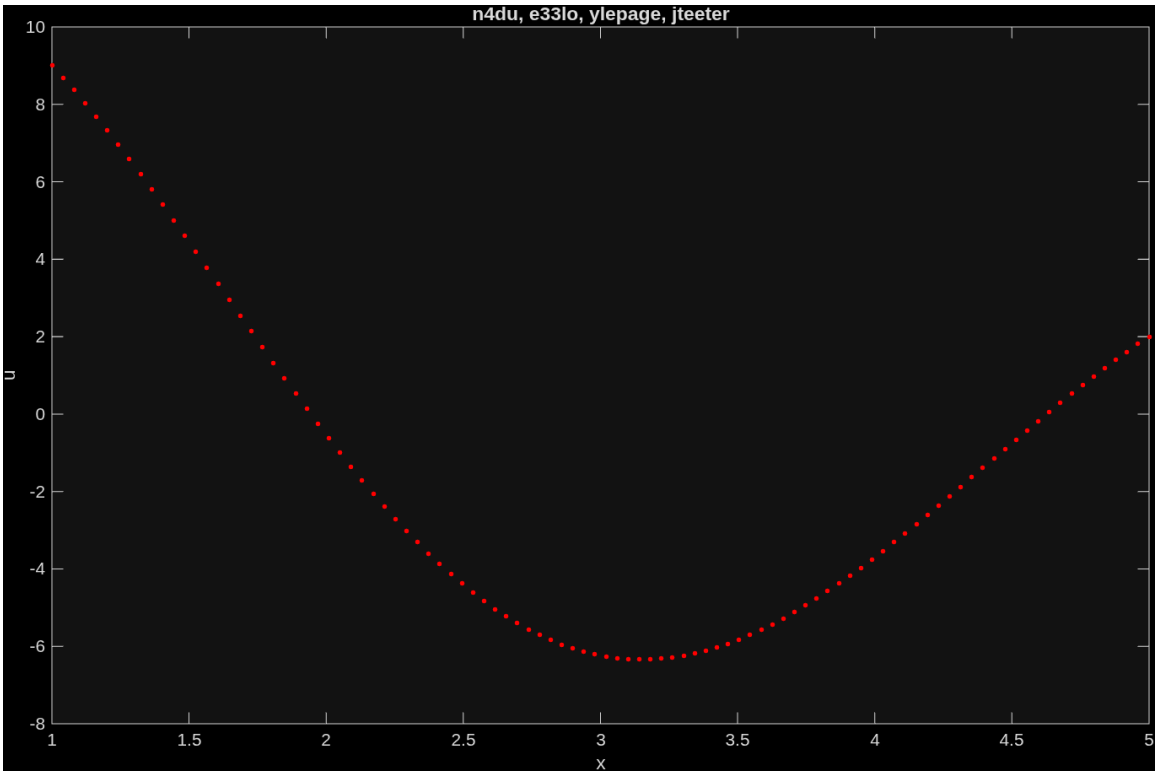

Figure 7. The approximation with 100 points.